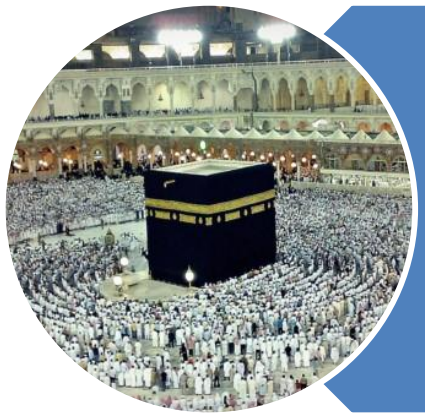
Data Structures - I

CPCS 204

رَبِّ اشْرَحْ لِي صَدْرِي وَيَسِّرْ لِي أَمْرِي وَاحْلُلْ عُقْدَةً مِنْ لِسَانِي يَفْقَهُوا قَوْلِي



O my Lord! Expand for me my chest [with assurance] and ease for me my task and untie the knot from my tongue that they may understand my speech. **(Quran 20 : 25-28)**

Dr. Jonathan (Yahya) Cazalas

Dr. Muhammad Umair Ramzan

CPCS 204

QUEUES

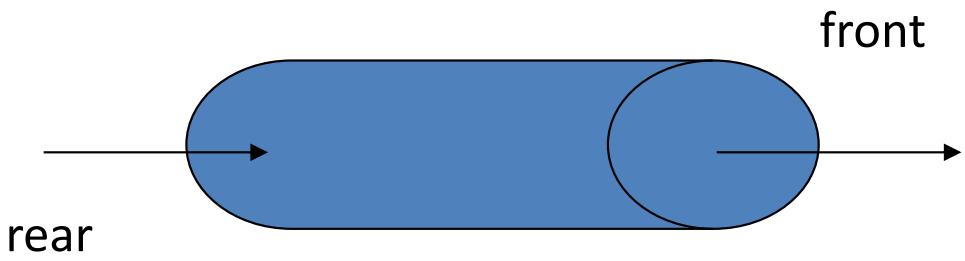
Implementation

Dr. Jonathan (Yahya) Cazalas

Dr. Muhammad Umair Ramzan

Implementation - Introduction

- Implementation
  - Arrays
  - Linked List
- Accessibility
  - Front & Rear, FIFO
- Operations
  - enqueue
  - dequeue
  - peek
  - isEmpty
  - isFull
  - clear



CPCS 204

Queues Implementation

Arrays

Dr. Jonathan (Yahya) Cazalas

Dr. Muhammad Umair Ramzan

Array Implementation

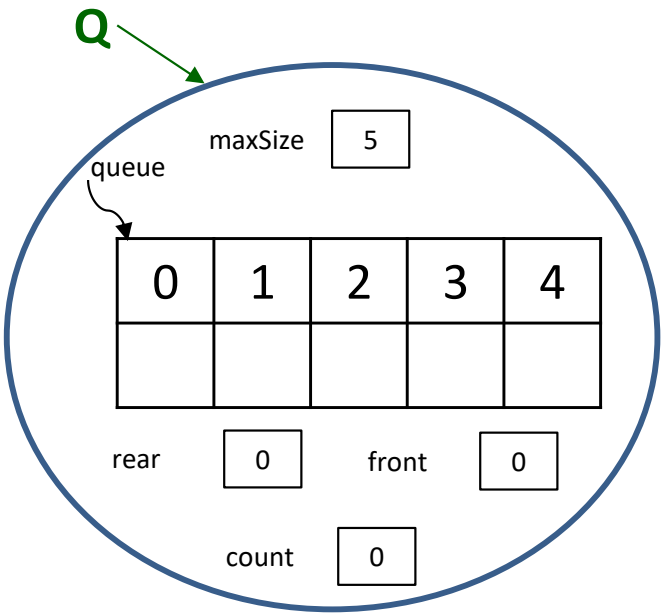
- Array
  - Insertion
    - first, last, Anywhere
  - Deletion
    - first, last, Anywhere
  - Traversal
  - Searching
  - Sorting
  - Merging
- Array as Queue
  - enqueue
    - Only at rear
  - dequeue
    - Only from front
  - peek
    - Return the value at front
  - isEmpty
    - count == 0
  - isFull
    - count == maxSize
  - clear
    - Remove all values of queues

CPCS 204

Queue Array - Class

```
class QueueArray {
    private int[] queue;
    private int front;
    private int rear;
    private int count;
    private int maxSize;
    // Constructor and methods
    public QueueArray(int size) {
        maxSize = size;
        queue = new int[maxSize];
        front = 0;
        rear = 0;
        count = 0;
    }
}
```

```
QueueArray Q;
Q = new QueueArray (5);
```



Dr. Jonathan (Yahya) Cazalas

Dr. Muhammad Umair Ramzan

Queues Array - **Methods**

- `public boolean isEmpty()`
- `public boolean isFull()`
- `public void enqueue(int value)`
- `public int dequeue()`
- `public int peek()`
- `public void clear()`

Queues Methods - `isEmpty()`

```
class QueueArray {
    private int[] queue;
    private int front;
    private int rear;
    private int count;
    private int maxSize;
    // Constructor and methods
    public QueueArray(int size) {
        maxSize = size;
        queue = new int[maxSize];
        front = 0;
        rear = 0;
        count = 0;
    }
}
```

```
public boolean isEmpty() {
    if (count == 0)
        return true;
    else
        return false;
}
```

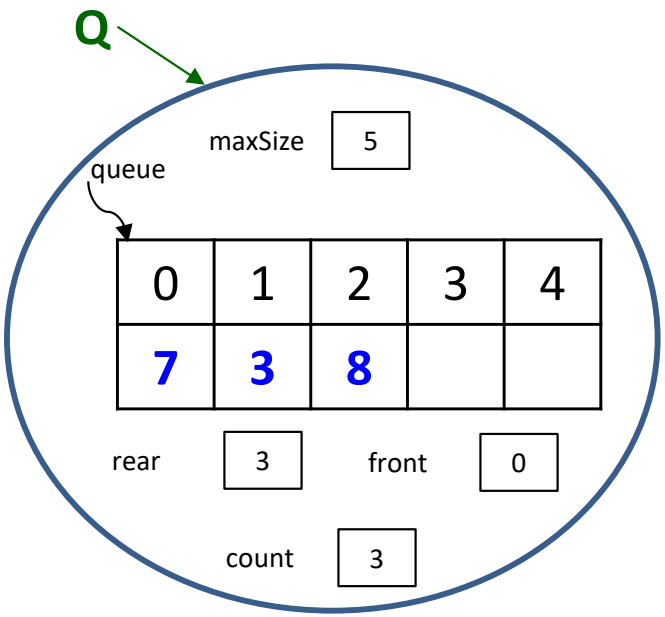
```
public boolean isEmpty() {
    return (count == 0);
}
```


Dr. Jonathan (Yahya) Cazalas

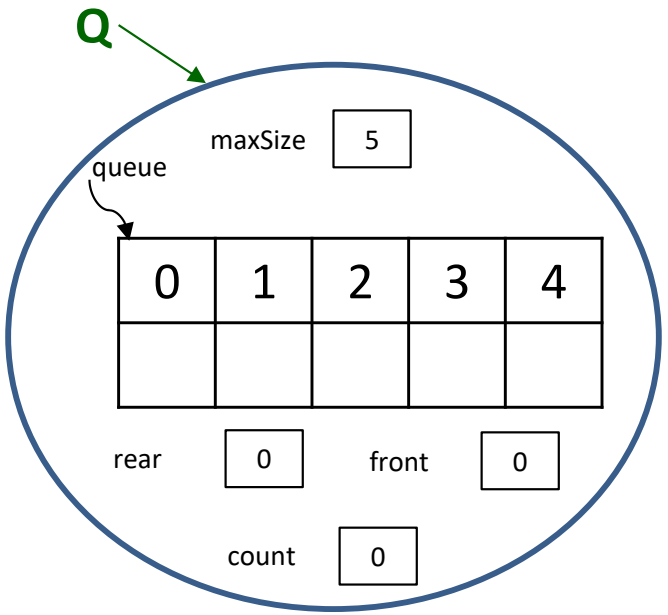
Dr. Muhammad Umair Ramzan

Queues Methods - `isEmpty()`

```
public boolean isEmpty() {
   return (count == 0);
}
```



False



True

Dr. Jonathan (Yahya) Cazalas

Dr. Muhammad Umair Ramzan

Queues Methods - `isFull()`

```
class QueueArray {
    private int[] queue;
    private int front;
    private int rear;
    private int count;
    private int maxSize;
    // Constructor and methods
    public QueueArray(int size) {
        maxSize = size;
        queue = new int[maxSize];
        front = 0;
        rear = 0;
        count = 0;
    }
}
```

```
public boolean isFull() {
    if (count == maxSize)
        return true;
    else
        return false;
}
```

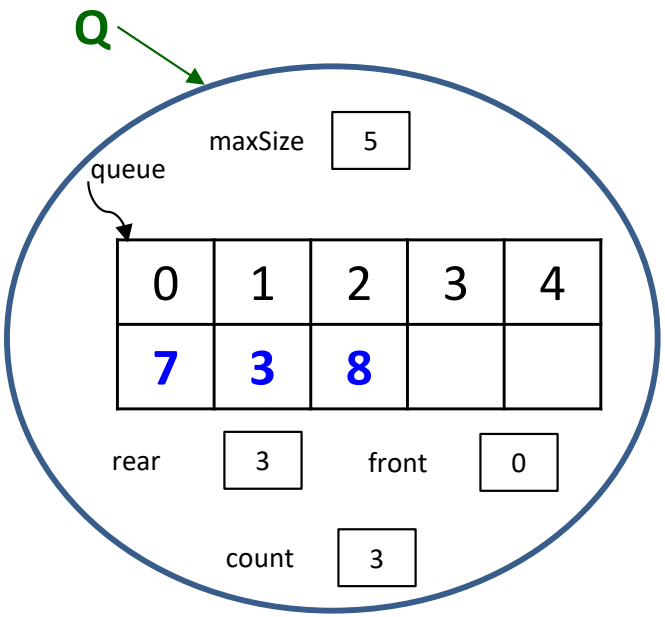
```
public boolean isFull() {
    return (count == maxSize);
}
```


Dr. Jonathan (Yahya) Cazalas

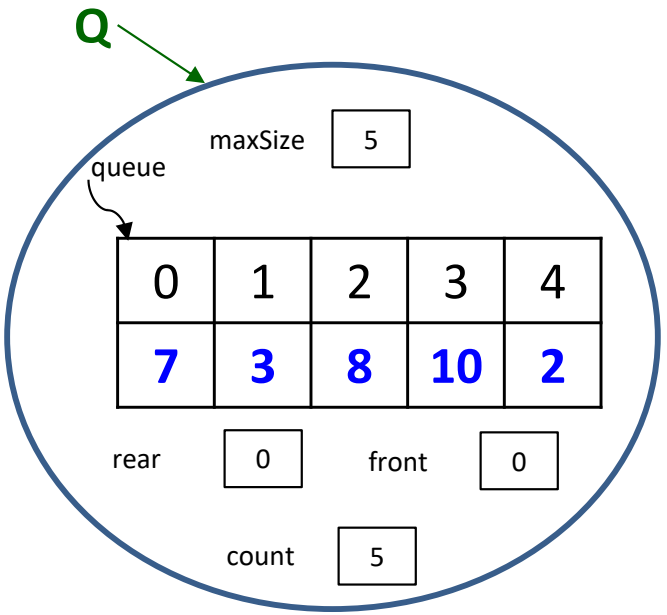
Dr. Muhammad Umair Ramzan

Queues Methods - `isFull()`

```
public boolean isFull() {
    return (count == maxSize);
}
```



False



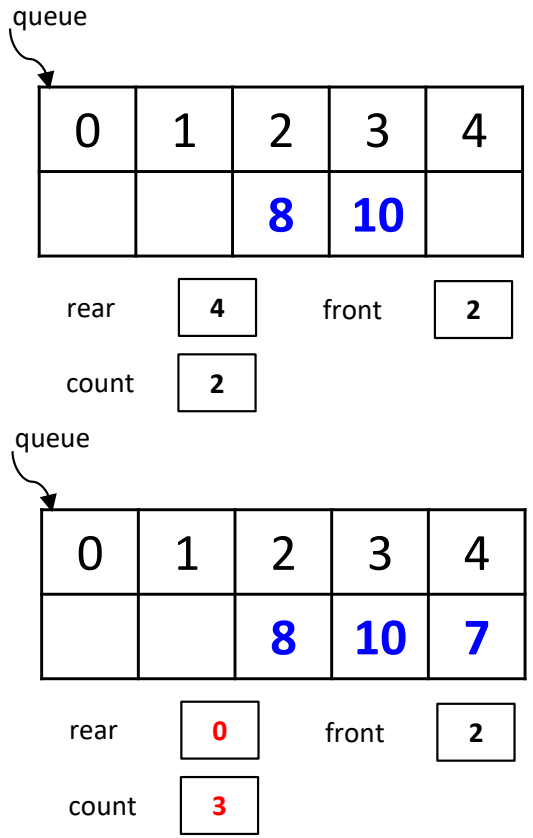
True

Dr. Jonathan (Yahya) Cazalas

Dr. Muhammad Umair Ramzan

Queues Methods - **enqueue**(value)

- Circular Array
- Check the queue is full or not
- Value will be store at "rear"
- Circular increment in "rear"
  - rear = (rear + 1) % maxSize;
- Linear increment in "count"
  - count++



rear = (4+1)%5

rear = 5%5 = 0

Dr. Jonathan (Yahya) Cazalas

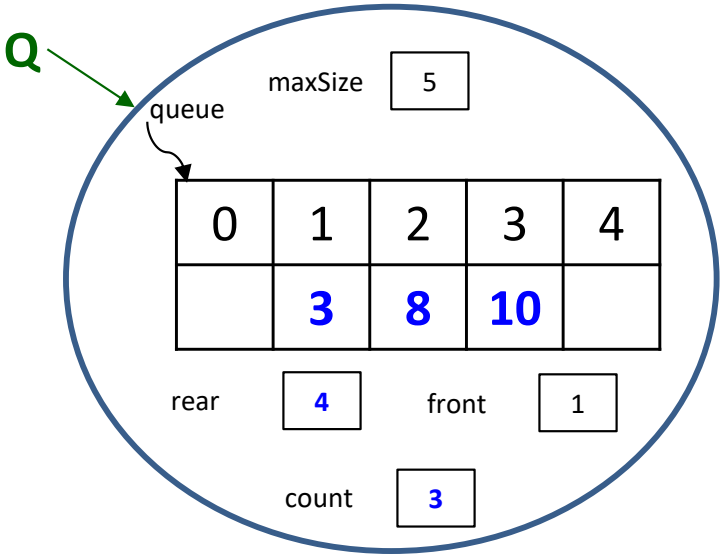
Dr. Muhammad Umair Ramzan

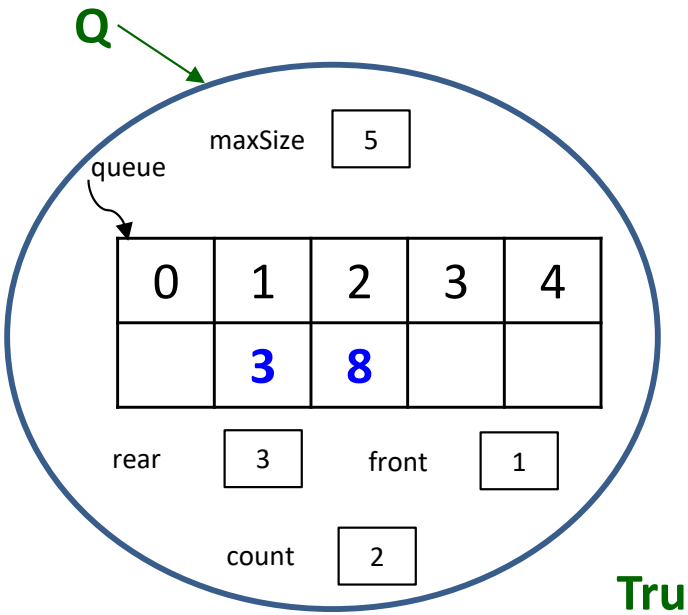
Queues Methods - **enqueue**(value)

```
public void enqueue(int value) {
    if (!isFull()){
       queue[rear] = value;
       rear = (rear + 1) % maxSize;
       count++;
    }
}
```

Queues Methods - **enqueue**(value)

```
public void enqueue(int value) {
    if (!isFull()){
        queue[rear] = value;
        rear = (rear + 1) % maxSize;
        count++;
    }
}
```





True

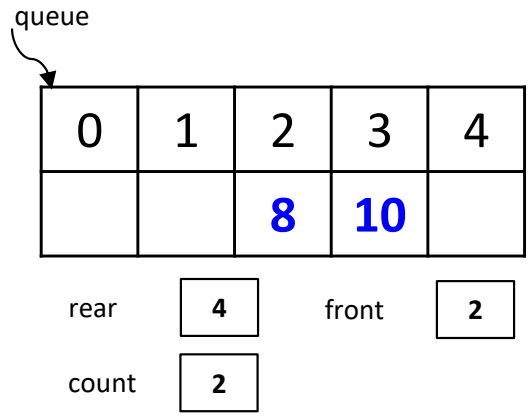
Dr. Jonathan (Yahya) Cazalas

Dr. Muhammad Umair Ramzan

CPCS 204

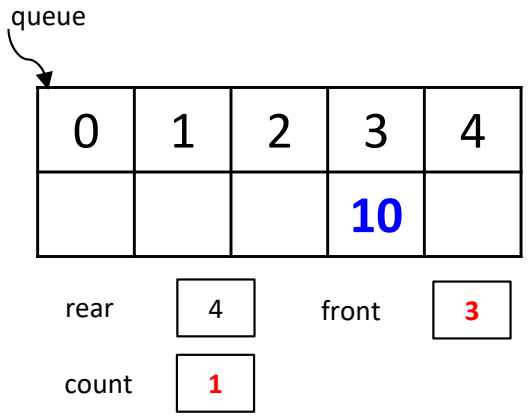
Queues Methods - dequeue ( )

- Circular Array
- Check the queue is empty or not
- Value will be removed from "front"
- Circular increment in "front"
  - front = (front + 1) % maxSize;
- Linear decrement in "count"
  - count--



front = (2+1)%5

front = 3%5 = 3



Dr. Jonathan (Yahya) Cazalas

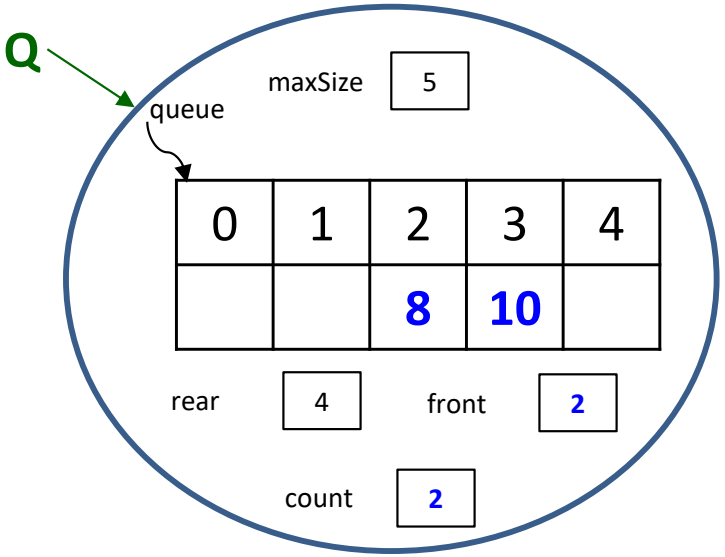
Dr. Muhammad Umair Ramzan

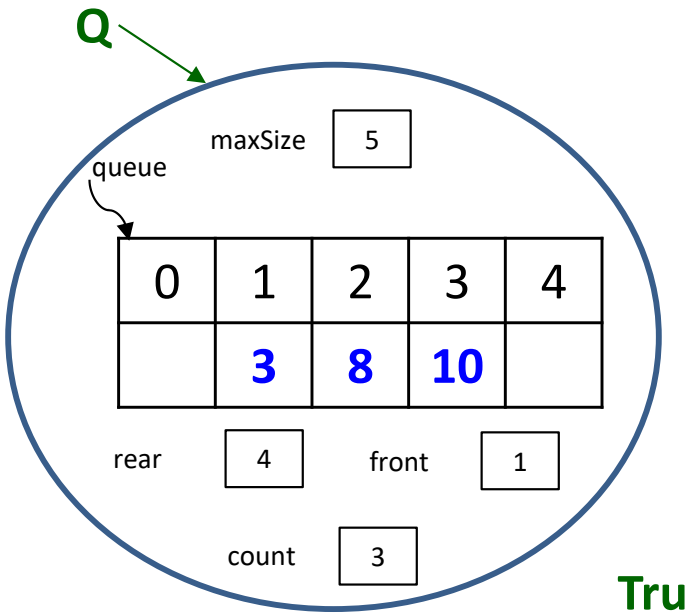
Queues Methods - **dequeue** ( )

```
public int dequeue( ) {
    if (!isEmpty()){
       int value = queue[front];
       front = (front + 1) % maxSize;
       count--;
       return value
    }
}
```

Queues Methods - dequeue ( )

```
public int dequeue( ) {
   if (!isEmpty()){
     int value = queue[front];
     front = (front + 1) % maxSize;
     count--;
     return value
   }
}
```





True

Value = 3

Dr. Jonathan (Yahya) Cazalas

Dr. Muhammad Umair Ramzan

Queues Methods - `peek` ( )

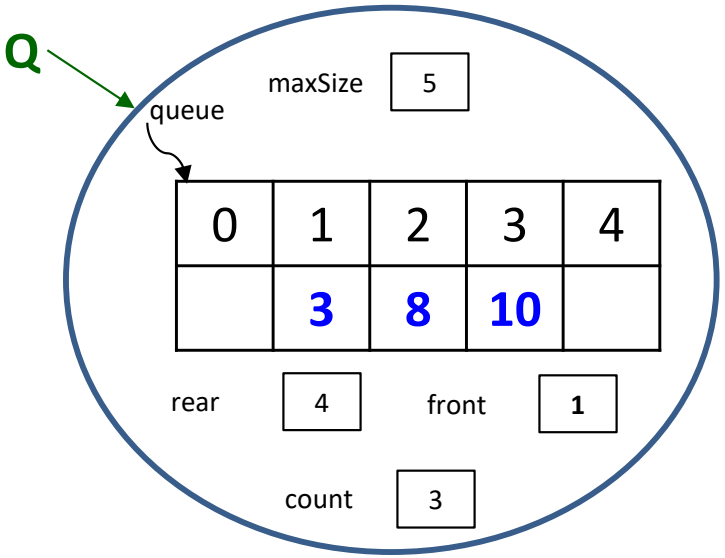
- Circular Array
- Check the queue is empty or not
- Value will be returned from "front"

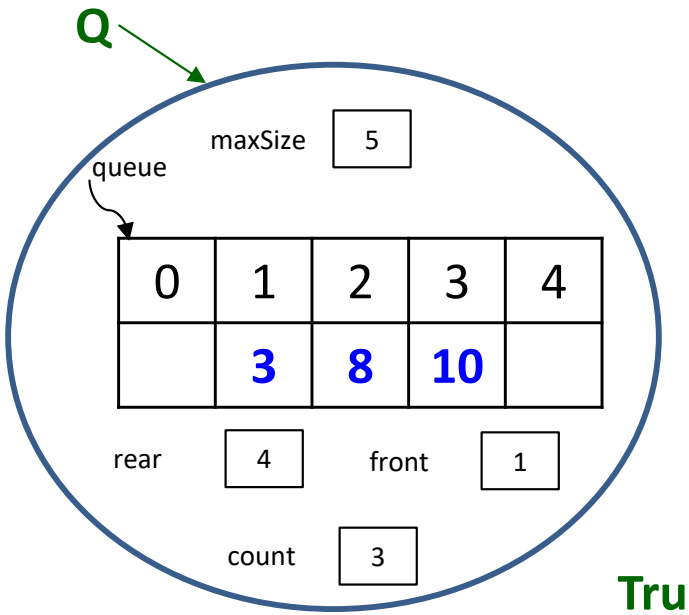
Queues Methods - peek ( )

```
public int peek( ) {
    if (!isEmpty()){
       return queue[front];
    }
}
```

Queues Methods - peek ( )

```
public int peek( ) {
    if (!isEmpty()){
       return queue[front];
    }
}
```





True

Value = 3

Dr. Jonathan (Yahya) Cazalas

Dr. Muhammad Umair Ramzan

Queues Methods - `clear()`

- Circular Array
- Check the queue is empty or not
- Apply "dequeue" repeatedly until queue become empty

Queues Methods - `clear` ( )

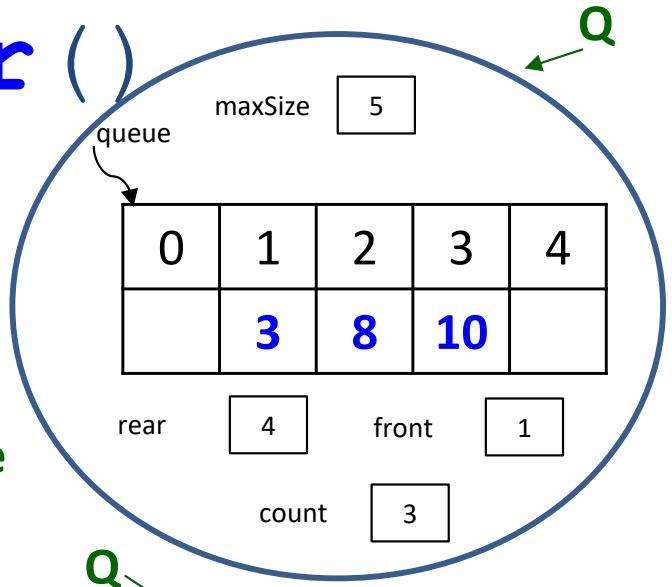
```
public void clear( ) {
    while (!isEmpty()){
       int value = dequeue();
       System.out.println(value);
    }
}
```

C P C S 2 0 4

Queues Methods - `clear()`

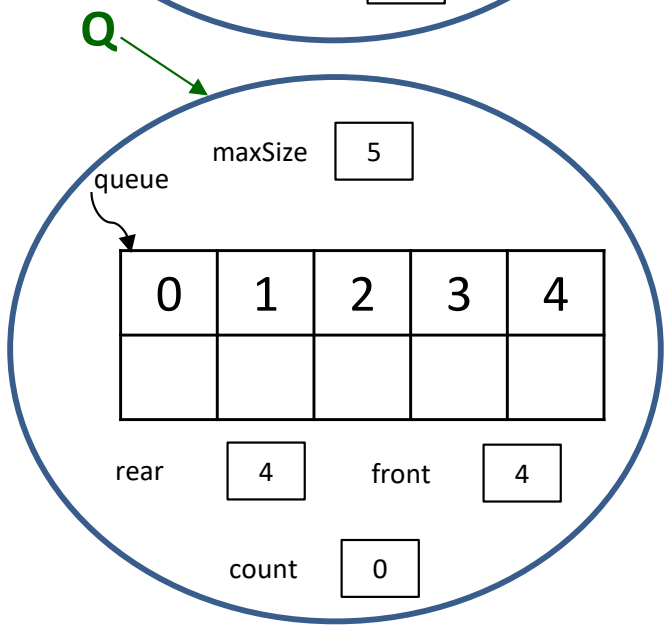
```
public void clear( ) {
   while (!isEmpty()){
      int value = dequeue();
      System.out.println(value);
   }
}
```

True



front	count
1	3
2	2
3	1
4	0

3 8 10



Dr. Jonathan (Yahya) Cazalas

Dr. Muhammad Umair Ramzan

CPCS 204

Queues Implementation

Linked List

Dr. Jonathan (Yahya) Cazalas

Dr. Muhammad Umair Ramzan

Linked List Implementation

- Linked List
  - Insertion
    - first, last, Anywhere
  - Deletion
    - first, last, Anywhere
  - Traversal
  - Searching
  - Merging
- Linked List as Queue
  - enqueue
    - Only at rear
  - dequeue
    - Only from front
  - peek
    - Return the value at front
  - isEmpty
    - count == 0
  - clear
    - Remove all values of queues

Queue Linked List – Node Class

```
public class QueueNode {
    private int data;
    private QueueNode next;

    public QueueNode(int i) {
          data = i;
          next = null;
    }
    public QueueNode(int i, QueueNode n) {
          data = i;
          next = n;
    }
}
```

Queue Linked List – Queue LL Class

```
public class QueueLL {
    private QueueNode front;
    private QueueNode rear;
    public QueueLL() {
        front = null;
        rear = null;
     }

    // POP, PUSH, TOP and more...
}
```

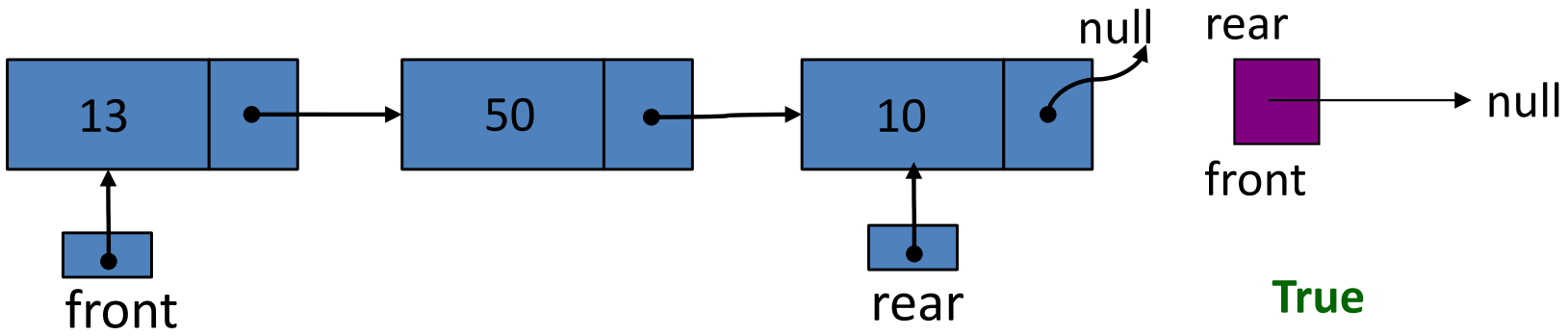
Queues LinkedList - **Methods**

- `public boolean isEmpty()`
- `public void enqueue(int value)`
- `public int dequeue()`
- `public int peek()`
- `public void clear()`

CPCS 204

Queues Methods - `isEmpty()`

```
public boolean isEmpty() {
    return front == null;
}
```

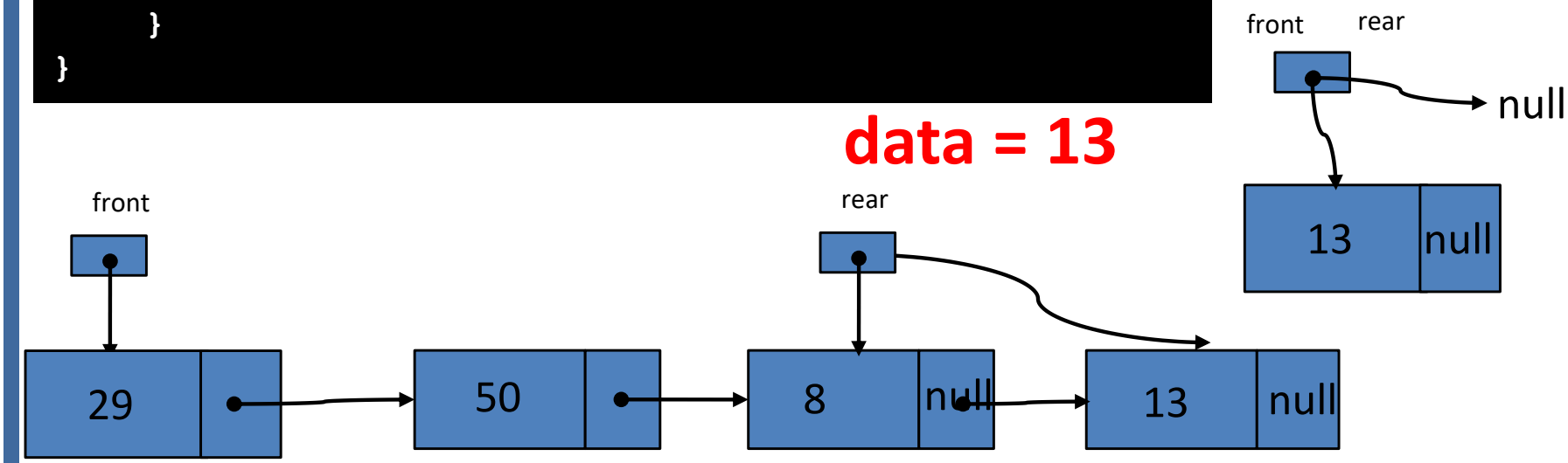


Dr. Jonathan (Yahya) Cazalas

Dr. Muhammad Umair Ramzan

Queues Methods - **enqueue** (value)

```
public void enqueue(int data) {
    if (front == null){
        front = new QueueNode (data, front);
        rear = front;
    }
    else{
        rear.next = new QueueNode(data, rear.next);
        rear = rear.next;
    }
}
```



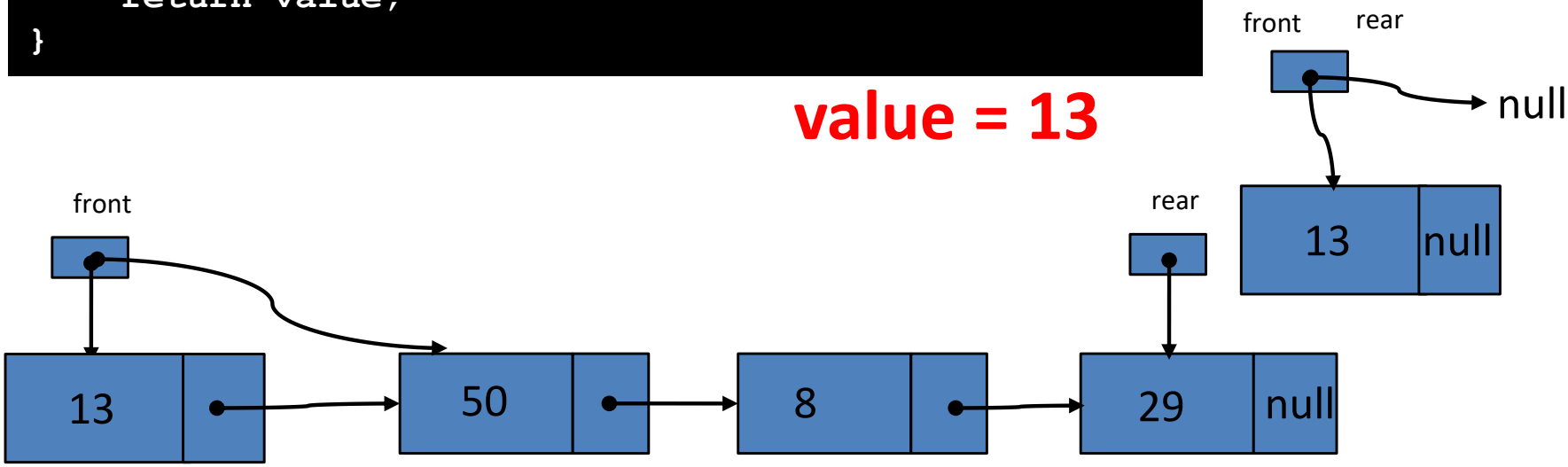
Dr. Jonathan (Yahya) Cazalas

Dr. Muhammad Umair Ramzan

Queues Methods - dequeue ( )

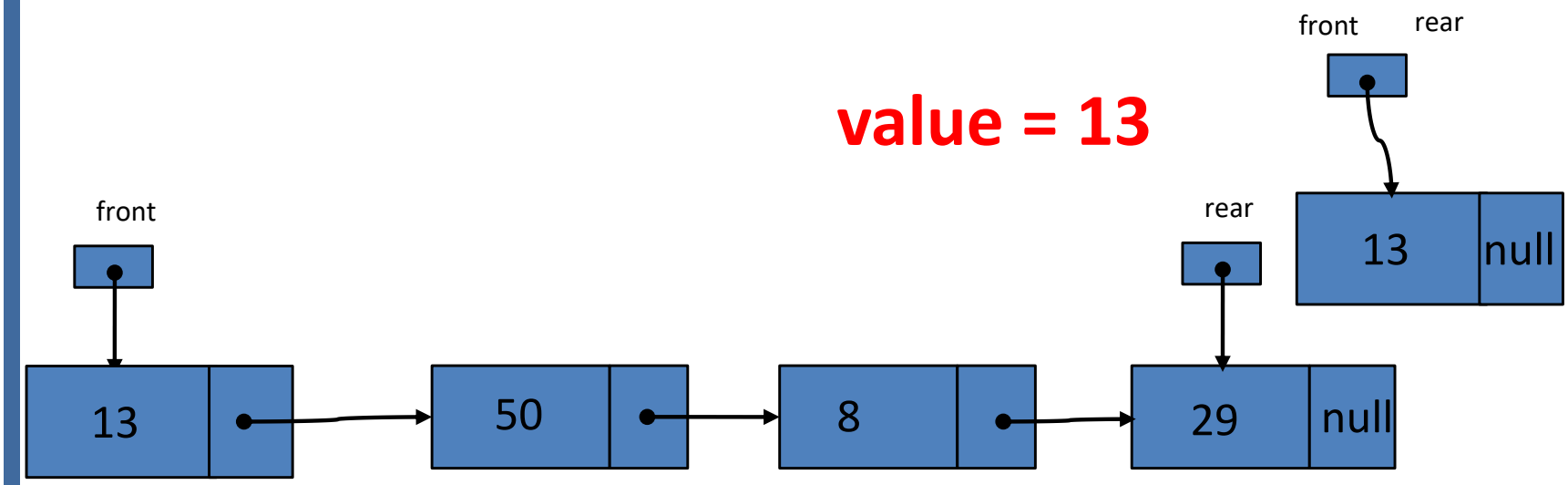
```
public int dequeue( ) {
    if ( !isEmpty() ){
        int value = front.data;
        if ( front.next == null ){
            front = front.next;
            rear = rear.next;
        }
        else
            front = front.next;
    }
    return value;
}
```

value = 13



Queues Methods - peek ( )

```
public int peek( ) {
   if ( !isEmpty() ){
      return front.data;
}
```



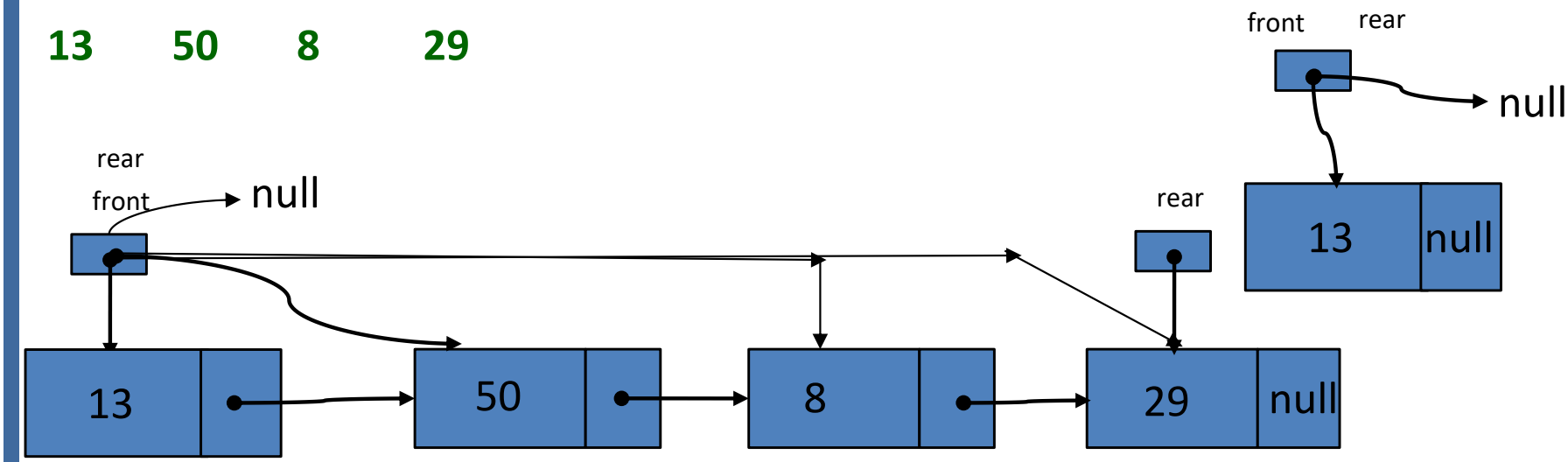
Queues Methods - `clear()`

```
public void clear( ) {
     while (!isEmpty()){
          int value = dequeue();
          System.out.println(value);
     }
}
```

13

Printing values

13 50 8 29



CPCS 204

Review Questions
Queues

Dr. Jonathan (Yahya) Cazalas

Dr. Muhammad Umair Ramzan

CPCS 204

MCQ

- In general, a queue Abstract Data Type allows:
  a) Insertions and removals anywhere.
  b) Insertions and removals only at one end.
  c) Insertions at the back and removals from the front.
  d) None of the above.

MCQ

- Suppose a queue of size **40** is implemented using a circular array. How many items are in the queue of the **front** is index **38** and the **rear** is index **9**? *(front & rear both were initialized with "0")*

a) 10
b) 11
c) 12
d) 13

CPCS 204

MCQ

- Suppose a queue of size 30 is implemented using a circular array. How many items are in the queue of the front is index 27 and the rear is index 6? *(front & rear both were initialized with "0")*

a) 9
b) 10
c) 11
d) 12

CPCS 204

MCQ

- Suppose a queue of size 50 is implemented using a circular array. How many items are in the queue if the front is at index 45 and the rear is at index 4? *(front & rear both were initialized with "0")*

a) 9
b) 10
c) 11
d) 12

CPCS 204

MCQ

- Which of the following is the most appropriate way to implement a queue using a singly-linked list?
  a) Enqueue at the head, dequeue at the tail.
  b) Enqueue at the tail, dequeue at the head.
  c) Both enqueue and dequeue at the head.
  d) Both enqueue and dequeue at the tail.

MCQ

- **Which of the following is the property of "queue"?**
  a) Last in, Last Out
  b) First in, First Out
  c) Accessed by both ends
  d) May be implemented using Arrays
  e) All of above

CPCS 204

MCQ

- **Which of the following is not an "operation" of the queue?**
  a) Enqueue
  b) Dequeue
  c) Peek
  d) isFull
  e) Clear
  f) None

CPCS 204

MCQ

- **Which of the following is kept movable while implementing queue using "brute force" method?**
  a) Front
  b) Rear
  c) Both
  d) None

CPCS 204

MCQ

- **Which of the following is kept movable while implementing queue using "circular array"?**
  a) Front
  b) Rear
  c) Both
  d) None

MCQ

- **How much time it will take to apply "clear( )" operation on queue?**
  a) O(1)
  b) O(n)
  c) O(log n)
  d) It is not an operation of queue

MCQ

- A circular queue is implemented using an array of size 10. The array index starts with 0, front is 6, and rear is 9. The insertion of next element takes place at the array index.
  a) 0
  b) 7
  c) 9
  d) 10

Tracing Question

- **(12 points) Queues and Stacks.** Let **Q** be a queue and S be a stack. The methods dequeue and pop obey the convention that they return whatever they remove. Assume that Q and S are initially empty and that i has been declared as an int. What would be printed by the following code fragment (**put answer in the box**)?
- Also, tell us the final contents of the queue and stack after execution of the code fragment.

Tracing Question

i = 0 1 2 3 4

```
Q.enqueue(10);
S.push(Q.front());
Q.enqueue(20);
S.push(Q.dequeue());
Q.enqueue(30);
  for(i = 0; i < 4; i++){
      System.out.print(Q.dequeue() + " ");
      System.out.print(S.pop() + " ");
      Q.enqueue(S.top() + i);
      S.push(Q.front());
  }
```

OUTPUT:

20 10 30 30 10 10 11 11

Final Queue Content:

12 13

Final Stack Contents

12
10

10	20	30	10	11	12	13			

12
11
10
30
10
10

Tracing Question

- **(12 points) Queues and Stacks.** Let **Q** be a queue and S be a stack. The methods dequeue and pop obey the convention that they return whatever they remove. Assume that Q and S are initially empty and that i has been declared as an int. What would be printed by the following code fragment (**put answer in the box**)?
- Also, tell us the final contents of the queue and stack after execution of the code fragment.

Tracing Question i = 10 20 30 40

```
S.push(50);
S.push(60);
Q.enqueue(30);
Q.enqueue(S.peek());
S.push(Q.front());
S.push(Q.dequeue());
Q.enqueue(20);

for(i = 10; i < 40; i += 10){
      System.out.print(S.pop() + " ");
      System.out.print(Q.dequeue + " ");
      S.push(Q.front() + i);
      Q.enqueue(S.peek() + 10);
}
```

OUTPUT:

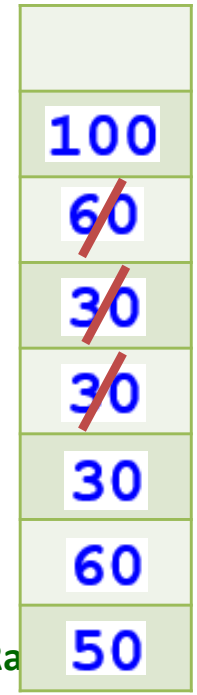
30 60 30 20 60 40

Final Queue Content:

70 110

Final Stack Contents

100
30
60
50



CPCS 204

Tracing Question

- (10 points) **Queues & Stacks.** Let **Q** be a queue and S be a stack. The methods dequeue and pop obey the convention that they return whatever they remove. Assume that Q and S are initially empty and that counter has been declared as an int. What would be printed by the following code fragment (**put answer in the box**)?
- Also, tell us the final contents of the queue and stack after execution of the code fragment.

Tracing Question

counter = 0 1 2 3 4 5

```
Q.enqueue(40);
S.push(Q.front());
Q.enqueue(60);
S.push(Q.dequeue());
Q.enqueue(30);
for(counter = 0; counter < 5; counter++){
    System.out.print(Q.dequeue() + " ");
    System.out.print(S.pop() + " ");
    Q.enqueue(S.top() + counter);
    S.push(Q.front());
}
```

OUTPUT:

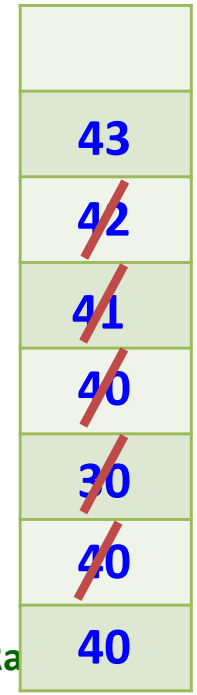
60 40 30 30 40 40 41 41 42 42

Final Queue Content:

43 44

Final Stack Contents

43
40



Coding Question

- (12 points) **Queue/Stack Code.** Assume that you have an existing **queue** of int values, ***Q***, and you also have an initially empty **stack** of int values, ***S***.
- Write a method, copyOddValues, that will copy all *odd* data values of the queue (from beginning to end) into the stack. The queue should then return to its original form. You should use the enqueue, dequeue, push, and pop operations of queues and stacks.

Coding Question

- EXAMPLE

Before running method

Queue: [12, 4, 7, 2, 11, 9, 6, 15, 3, 8]

Stack: [ ]

After running method

Queue: 12, 4, 7, 2, 11, 9, 6, 15, 3, 8

Stack: 3, 15, 9, 11, 7

Solution

```
public static void copyOddValues(Queue Q, Stack S) {
    int temp;
    int i = 0;
    while (!Q.isEmpty()) {
            temp[i] = Q.dequeue();
            if (temp[i]%2 == 1)
                S.push(temp[i]);
            i++;
    }
    for ( int n=0; n < i; n++)
         Q.enqueue(temp[n]);
}
```

Coding Question

- (12 points) **Stack Code. Queue/Stack Code.** Assume that you have an existing **queue** of int values, ***Q***, and you also have an initially empty **stack** of int values, ***S***.
- Write a method, moveEvenValues, that will move all *even* data values of the queue (from beginning to end) into the stack. The queue should retain the *odd* data values when the method is over. You should use the enqueue, dequeue, push, and pop operations of queues and stacks.

Coding Question

- EXAMPLE

Before running method

Queue: [12, 4, 7, 2, 11, 9, 6, 15, 3, 8]

Stack: [ ]

After running method

Queue: [7, 11, 9, 15, 3]

Stack: [12, 4, 2, 6, 8]

Solution

```
public static void moveEvenValues(Queue Q, Stack S) {
        int i = 0;
        int[] a;
        while (!Q.isEmpty()){
                int temp = Q.dequeue( );
                if ( temp % 2 == 0 )
                    S.push (temp);
                else
                    a[i++] = temp;
        }
        for (int n=0; n<i; n++)
            Q.enqueue(a[n]);
}
```

Coding Question

- Write a method which returns the **maximum value** from the data stored in a **queue**. You may use the standard functions like enqueue( ) and dequeue( ) etc for manipulation of the queue. Before returning the **maximum value**, the queue must be in its original status.

Solution

```
public static int maxValue(Queue Q) {
        int i = 0;
        int[] a, max = -32767;
        while (!Q.isEmpty()){
              int temp = Q.dequeue( );
              if ( temp > max )
                   max = temp;
              a[i++] = temp;
        }
        for (int n=0; n<i; n++)
             Q.enqueue(a[n]);
        return max;
}
```

Coding Question

- **Queue/Stack Code.** Assume that you have an existing **stack** ***S***, of int values.
- Write a method, **distributeStackValues**(), that will move all ***values which are greater than 50*** to queue and ***remaining values*** to another queue. The method should then **print data values of the two queues**. Assume that the enqueue, dequeue, push, and pop etc. operations of queues and stacks are available. You should create two queues to accommodate the values of defined stack.

Coding Question

- EXAMPLE

Before running method

Stack: [100, 40, 60, 88, 11, 95, 73, 15, 3, 8]

100: is on top of stack

After running method

Values in queue1: 100, 60, 88, 95, 73

Values in queue2: 40, 11, 15, 3, 8

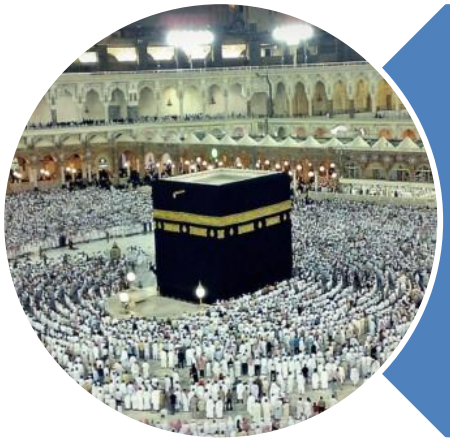
Solution

```
public static void distributeStackValues (Stack stack) {
        Queue q1 =new Queue();
        Queue q2 =new Queue();
        While(!stack.isEmpty()){
            int value= stack.pop();
            if(value>50)
               q1.enQueue(value);
            else
               q2.enQueue(value);
        }
        System.out.print("Values in Queue1: ");
        While(!q1.isEmpty())
              System.out.print(q1.deQueue()+ ", ");
        System.out.println();
        System.out.print("Values in Queue2: ");
        While(!q2.isEmpty())
              System.out.print(q2.deQueue()+ ", ");
        System.out.println();
}
```

Data Structures - I

"No Bonus" marks for this course anymore

CPCS 204



Allah (Alone) is Sufficient for us, and He is the Best Disposer of affairs (for us).(Quran 3 : 173)

Dr. Jonathan (Yahya) Cazalas

Dr. Muhammad Umair Ramzan